

12. Dataset Description

12.1 Provenance

The dataset used to train and evaluate this system is synthetic. It contains 22,000 records across 30 columns and was produced by `data/raw/generate_dataset.py` under a fixed random seed of 42. No survey was administered, and no row corresponds to a real person.

This should be stated at the outset because it constrains how every result in Section 23 must be read. The model's reported accuracy measures how well a learning algorithm recovers a generating process that is known in advance. It does not measure how well the system would classify the risk profile of an actual individual, and no claim to that effect is made anywhere in this report.

Synthetic generation was chosen for a practical reason rather than a methodological preference. The system requires a single table combining demographic attributes, self-reported financial position, lifestyle spending and a behavioural risk label. No public dataset carries all four. Financial datasets such as credit-default records lack lifestyle and behavioural variables; behavioural finance studies typically report aggregate findings rather than record-level data; and datasets that combine both are proprietary. Constructing a generator with explicitly documented dependencies at least makes the assumptions inspectable, which a poorly documented real dataset would not.

12.2 Schema

The thirty columns divide into four groups.

Group	Count	Columns
Demographic	10	age, gender, education, occupation, income_level, marital_status, dependents, location, employment_type, years_of_experience
Financial	10	monthly_income, monthly_expenses, savings_rate, emergency_fund_months, total_debt, loan_amount, credit_score, investment_experience_years, investment_frequency, insurance_coverage
Lifestyle	6	shopping_frequency, online_spending_pct, luxury_spending_pct, subscription_count, gaming_expenses_monthly, travel_expenses_annual
Target	4	risk_category, investment_preference, expected_savings_increase, financial_decision_score

Only `risk_category` is used as the classification target in the deployed model. `investment_preference` supports a secondary model discussed in Section 22. `financial_decision_score` and `expected_savings_increase` are excluded from the feature set entirely, since both are derived from the same quantities that determine the risk label and their inclusion would constitute leakage.

12.3 Dependency Structure

Attributes are not drawn independently. The generator imposes a chain of conditional dependencies intended to produce plausible joint behaviour rather than a table of uncorrelated noise.

Age is sampled from a normal distribution centred at 35 and clipped to 18–75. Occupation is drawn from a base distribution, then overridden for respondents under 24 so that students dominate that band. Income level is conditioned on occupation; a business owner draws `High` with probability 0.65, a student draws `Low` with probability 0.80. Location is conditioned on income level, so that high earners concentrate in urban areas. Monthly income is a product of an occupation base rate, an education multiplier ranging from 0.8 for high school to 1.6 for a doctorate, and a lognormal dispersion term.

Expenses follow income through a ratio drawn uniformly between 0.40 and 0.90 and inflated by 0.04 per dependant. Savings rate is then the implied surplus with Gaussian noise added, which is why the column occasionally goes negative. Credit score is constructed as a linear function of debt-to-income, savings rate and years of experience, with noise of standard deviation 40. Investment experience scales with tenure and an internal discipline variable. Investment frequency and insurance coverage are both sampled from probability vectors that depend on that same discipline variable.

A consequence of this design is that many features are strongly correlated with the targets by construction. Measured against `financial_decision_score`:

Feature	Pearson <i>r</i>
savings_rate	+0.711
credit_score	+0.499
emergency_fund_months	+0.482
investment_experience_years	+0.346
debt_to_income ratio	−0.223

These are not discovered relationships. They are the relationships the generator was written to produce, recovered by measurement.

12.4 Construction of the Target Labels

`risk_category` derives from a continuous score:

```
risk_score = -30·savings_rate
             + 25·(1 - credit_score/900)
             + 20·(min(debt_to_income, 5)/5)
             - 15·(investment_experience_years/20)
             + 10·[emergency_fund_months < 3]
             + ε,      ε ~ N(0, 8)
```

The score is cut at its 35th and 80th percentiles to yield Low, Medium and High in proportions 35 / 45 / 20.

The noise term deserves attention. The deterministic component of `risk_score` has a standard deviation of 7.64 across the dataset; the injected noise has a standard deviation of 8.00. The noise is therefore slightly larger than the signal. Roughly half the variance in the continuous score — and hence a substantial share of the class assignment near the two thresholds — is irreducible. Section 22.2 derives the resulting Bayes-optimal accuracy of 0.6104 and shows that the trained model reaches 98.9% of it.

`investment_preference` is generated differently, and the difference is consequential. Each record's preference is sampled from one of three fixed probability vectors selected solely by that record's `risk_category`. A Low-risk record draws Fixed Deposit with probability 0.40, Crypto with probability 0.05, and so on. **No other attribute influences the draw.** The label consequently carries no information about the twenty-six input features beyond what `risk_category` already encodes, which is why the secondary model fails. Section 22 develops this.

`financial_decision_score` is a weighted sum of savings rate, normalised credit score, inverted debt burden, investment experience and emergency-fund coverage, plus noise. `expected_savings_increase` scales the internal discipline variable.

12.5 A Defect in the Data Loading Path

One column does not contain what the generator intended, and the discrepancy survived into the trained model.

The generator emits `insurance_coverage` as a three-level attribute. Reading the CSV file as raw text confirms the intended distribution:

Value on disk	Count	Share
Basic	9,727	44.2%
None	7,264	33.0%
Comprehensive	5,009	22.8%

`pandas.read_csv` includes the bare string `None` in its default set of null indicators. When `preprocess.py` loads the file, those 7,264 entries arrive as `NaN`. The script identifies them as missing values, reports them as such, and imputes them with the column mode — which is `Basic`.

The effect is that a third of the dataset had a genuine, meaningful category replaced by a different category. The fitted ordinal map contains two levels rather than three, and no record in the training set carries the information that its subject held no insurance at all.

The consequences are uneven, and it is worth separating them.

On predictive accuracy the effect is negligible. Retraining with the category preserved gives a weighted F1 of 0.5915 against 0.5955 for the pipeline as shipped, a difference smaller than the ± 0.0032 cross-validation spread reported in Section 23. The reason is structural: `insurance_coverage` does not appear anywhere in the `risk_score` expression above. It influences the label only indirectly, through the internal discipline variable, and that pathway is already carried by `savings_rate` and `emergency_fund_months`, both of which the model observes directly. The classifier loses nothing it was using.

On the derived scores and on serving, the effect is real. `behavioral_composite_score` assigns insurance coverage a weight of 0.20, so that score is systematically overstated for the third of records whose coverage was upgraded from none to basic. And because the ordinal map never learned a `None` level, a live user who selects "no insurance" in the questionnaire is silently encoded as the training median. The system accepts an input it cannot represent.

The defect is documented rather than repaired. Repairing it would shift every metric in this report by less than half a percentage point while invalidating the figures already generated, and the finding is more instructive left in place: it illustrates that a data-quality fault can pass through cleaning, encoding, training and evaluation without ever registering as an error, because the offline test set is corrupted in precisely the same way as the training set. Section 26 records the limitation.

12.6 Summary Statistics

Column	Min	Median	Max
monthly_income (₹)	5,000	45,802	200,000
monthly_expenses (₹)	3,000	30,212	180,000
savings_rate (fraction)	-0.100	0.319	0.600
credit_score	300	791	900
total_debt (₹)	13	10,717	4,166,885
financial_decision_score	0.00	54.20	85.34

Note that `savings_rate` is stored as a fraction, not a percentage. The public API accepts this field as a percentage in the range 0–100 and converts it before inference. The absence of that conversion in an earlier build was one of the defects discussed in Section 22.

Figure 12.1 — *Class distribution of `risk_category`*. Already generated at `data/processed/plots/risk_category_distribution.png`. Place in Section 12.4.

Figure 12.2 — *Correlation heatmap of the numeric features*. Already generated at `data/processed/plots/correlation_matrix.png`. Use it to support the correlation table in Section 12.3. Place there.

Figure 12.3 — *Distribution of the continuous `risk_score` with the 35th and 80th percentile cut points overlaid, and the $N(0, 8)$ noise band shaded*. Not yet generated. Add a plotting block to `generate_dataset.py` after the `risk_score` array is computed. This is the single most useful figure in the report, because it makes the irreducible class overlap visible rather than merely asserted. Place in Section 12.4 immediately after the `risk_score` equation.

Table 12.1 — *Full column dictionary*. Thirty rows: name, type, group, permitted values or range, and whether the column is used as a model feature. Derive from `data/raw/dataset_description.md` and `ml/models/feature_columns.json`. Place as an appendix and reference it from Section 12.2.